

CHAPTER 16 • APPLIED INFRASTRUCTURE AS CODE

Real-World Projects & Hands-On Labs

Four end-to-end builds: a highly available web app, a Kubernetes cluster, a CI/CD pipeline, and a multi-cloud deployment.



AWS HA App



Kubernetes



CI/CD



Multi-Cloud

OVERVIEW

Four hands-on projects

Each project applies core Terraform concepts to a scenario you'll actually run in production.



01

Highly Available Web App in AWS

EC2, Auto Scaling, ALB, and RDS working together.



02

Kubernetes Cluster with Terraform

Provision EKS and deploy a sample Nginx app.



03

Terraform in a CI/CD Pipeline

Automate plan & apply with GitLab CI/CD.



04

Multi-Cloud Setup

Deploy identical infrastructure to AWS & Azure.

Deploy a scalable web app on AWS

A production pattern combining compute, traffic distribution, elasticity, and a managed database.



EC2 Instances

The application servers running your code.



Application Load Balancer

Distributes incoming traffic across healthy instances.



Auto Scaling Group

Adds or removes instances to match demand.



RDS Database

A managed, separate data layer for durability.

Steps 1-2 — Provider, launch template, ASG & ALB

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_launch_template" "web" {  
  name_prefix   = "web-template"  
  image_id      = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
}  
  
resource "aws_autoscaling_group" "web" {  
  vpc_zone_identifier = ["subnet-xxxxx"]  
  desired_capacity    = 2  
  max_size            = 5  
  min_size            = 1  
  launch_template {  
    id       = aws_launch_template.web.id  
    version = "$Latest"  
  }  
}
```

```
resource "aws_lb" "web_lb" {  
  name                = "web-alb"  
  internal            = false  
  load_balancer_type = "application"  
  security_groups     = ["sg-xxxxx"]  
  subnets            = ["subnet-xxxxx"]  
}
```



Why this works

The launch template defines what each instance looks like. The Auto Scaling Group keeps 1-5 of them running based on demand, and the ALB spreads traffic across whichever instances are healthy — together, this removes any single point of failure.

Step 3 — Deploy the RDS database



```
resource "aws_db_instance" "web_db" {  
  allocated_storage = 20  
  engine            = "mysql"  
  instance_class    = "db.t2.micro"  
  username          = "admin"  
  password          = "password123"  
}
```



Why separate it

Running the database as its own RDS instance — instead of on an app server — lets it scale, back up, and fail over independently of the web tier.



Production checklist: never hard-code credentials as shown here — use a secrets manager or Terraform variables marked **sensitive** = **true**, and enable Multi-AZ for real failover.

Provision EKS & deploy a workload

Terraform doesn't stop at infrastructure — it can hand off directly into the Kubernetes API.



Provision the cluster

Use the `terraform-aws-modules/eks` module to stand up an EKS control plane and node groups.



Deploy a workload

Use the `kubernetes` provider to declare a Deployment — here, a 2-replica Nginx app.

Step 1 — Deploy an EKS cluster

```
module "eks" {  
  source      = "terraform-aws-modules/eks/aws"  
  cluster_name = "my-cluster"  
  cluster_version = "1.27"  
  subnets    = ["subnet-xxxxx"]  
  vpc_id      = "vpc-xxxxx"  
}
```

What this module gives you

- A managed EKS control plane
- Worker node groups wired to your VPC
- IAM roles & security groups pre-wired
- A pinned Kubernetes version (1.27)

Step 2 — Deploy the Nginx application

```
resource "kubernetes_deployment" "nginx" {  
  metadata {  
    name = "nginx"  
  }  
  spec {  
    replicas = 2  
    template {  
      metadata {  
        labels = {  
          app = "nginx"  
        }  
      }  
      spec {  
        container {  
          image = "nginx:latest"  
          name = "nginx"  
        }  
      }  
    }  
  }  
}
```



Result

Two Nginx pods come online behind the deployment, labeled app: nginx so a Service can target them.

Same workflow scales to any Helm-less workload — Deployments, Services, ConfigMaps, and Secrets are all native Terraform resources via the kubernetes provider.

Automate provisioning with GitLab CI/CD

A .gitlab-ci.yml with two stages: plan on every push, apply only after review.

```
stages:
  - plan
  - apply

terraform:
  image: hashicorp/terraform:latest
  script:
    - terraform init
    - terraform plan -out=tfplan
  only:
    - main
```

```
terraform_apply:
  image: hashicorp/terraform:latest
  script:
    - terraform apply -auto-approve tfplan
  only:
    - main
```



Both jobs run only on the **main** branch, and apply reuses the exact plan file from the plan stage — no drift between what was reviewed and what gets applied.

Deploy identical infrastructure to AWS & Azure

One configuration, two providers — built for redundancy and failover.

```
provider "aws" {  
  region = "us-east-1"  
}  
  
provider "azurerm" {  
  features {}  
}
```



Same app, deployed on both AWS and Azure — if one cloud has an outage, the other keeps serving traffic.

```
resource "aws_instance" "web_aws" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
}  
  
resource "azurerm_virtual_machine" "web_azure" {  
  name                 = "web-vm"  
  location              = "East US"  
  resource_group_name = "my-resource-group"  
  vm_size               = "Standard_B1s"  
}
```

Four projects, one consistent workflow



HA Web App

EC2 + Auto Scaling + ALB + RDS for a resilient AWS stack.



Kubernetes Cluster

Provisioned EKS and deployed Nginx with Terraform.



CI/CD Pipeline

GitLab automates terraform plan and apply on every merge.



Multi-Cloud

Identical infrastructure across AWS & Azure for failover.